



Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (`__init__`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

PEAS stands for Performance Measure, Environment, Actuators, and Sensors. Performance Measure checks how well the agent is doing. Environment is the place where the agent works. Actuators are used to do actions, and Sensors are

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

The environment is represented by variables such as `self.width`, `self.height`, `self.agent_pos`, `self.walls`, `self.food_positions`, `self.opponents`, and `self.toxic_traps`. These variables describe the grid, the agent position, walls, food, opponents, and

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

The environment is designed as a Multi-Agent environment because it can have opponents. The opponents can move and affect the main agent by colliding with it. In my current program, the number of opponents is set to 0, but the

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the full history of information received by the agent from the beginning up to the current time. The agent can use this information to decide what action to do next.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

The `get_percept()` method represents the Sensors part of PEAS. It gives information about the environment to the agent, such as its position, opponents, food, collisions, score, and toxic traps.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()` , is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

The environment is Partially Observable because the agent cannot see all the information in the environment. It can know its position, opponent positions, score, and some other details, but it cannot directly see all wall, food, and toxic

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

The Performance Measure is updated because the agent's score is reduced by 5 points when it hits a wall. The score is used to measure how well the agent is performing.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

The performance measure should check the real result in the environment, not only what the agent is doing internally. If we reward only actions, the agent may do unnecessary actions just to get more points. The score should show whether

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

The environment becomes Partially Observable because toxic traps are part of the environment, but the agent cannot see them. Therefore, the agent does not know the full state of the environment.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

smells_toxin gives the agent information about a dangerous toxic trap. Since a toxic trap reduces the score by 15 points, this information can help the agent avoid dangerous places and choose better actions. This can help the agent get a

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

In the Vacuum World example, the vacuum gets points every time it cleans dirt. A badly designed agent could clean the dirt, put the dirt back on the floor, and clean it again to keep getting points. This shows that the performance measure

Finalize and Lock Lab 01 Analytical Evaluation



FACULTY OF COMPUTING